

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Фронтенд разработка

Лабораторная работа 2 «Взаимодействие с внешним API»

Выполнили:

Лобова П.М.

К3344

Мельник С.Н.

К3342

Проверил:

Добряков Д.И.

2025 год

Санкт-Петербург

Задача:

Привязать ранее написанный сайт к внешнему API средствами fetch. Реализовать моковое API средствами JSON-сервера и подключить к нему авторизацию.

Выполнение:

Моковое API и авторизация

Для начала скачиваем node.js. После установки был настроен JSON-сервер, выступающий в качестве базы данных для моковых API и заполним его данными для работы с ресурсами (register, bookings, restaurants)

```
fetch("http://localhost:5502/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify(newUser)
})
.then(response => response.json())
.then(data => {
  console.log("Пользователь успешно зарегистрирован:", data);
  alert("Регистрация успешна! 🎉");

  document.getElementById("signupForm").reset();

  window.location.href = "login.html";
})
.catch(error => {
  console.error("Ошибка при регистрации:", error);
  alert("Что-то пошло не так. Попробуйте еще раз!");
});
});
```

Рис. 1 - Код страницы для регистрации для сохранения нового пользователя

```

try {
  const response = await fetch(`${API_URL}/bookings?userId=${user.id}`);
  const bookings = await response.json();

  const bookingHistoryDiv = document.getElementById("bookingHistory");
  if (bookings.length === 0) {
    bookingHistoryDiv.innerHTML = "<li>У вас нет истории бронирований.</li>";
    return;
  }

  for (const booking of bookings) {
    const restaurantResponse = await fetch(`${API_URL}/restaurants/${booking.restaurantId}`);
    const restaurant = await restaurantResponse.json();
    booking.restaurantName = restaurant.name || "Неизвестный ресторан";
  }

  bookingHistoryDiv.innerHTML = bookings.map(b => `
    <li>
      <h3>${b.date}, ${b.time}</h3>
      <p>Ресторан: <i>${b.restaurantName}</i></p>
      <p>Количество гостей: <i>${b.guests}</i></p>
      <p>Статус бронирования: <i>${b.status}</i></p>
    </li>
  `).join("");
} catch (error) {
  console.error("Ошибка загрузки бронирований:", error);
  document.getElementById("bookingHistory").innerHTML = "<li>Ошибка загрузки данных.</li>";
}

```

Рис. 2 - Код для отображения бронирований в разделе аккаунта пользователя

```

async function loadRestaurant() {
  const restaurantId = getRestaurantIdFromUrl();
  if (!restaurantId) {
    document.getElementById("restaurant-name").innerText = "Ресторан не найден";
    return;
  }

  try {
    const response = await fetch(`${API_URL}/restaurants?id=${restaurantId}`);
    if (!response.ok) throw new Error("Ошибка загрузки данных");

    const data = await response.json();
    if (data.length === 0) {
      document.getElementById("restaurant-name").innerText = "Ресторан не найден";
      return;
    }

    const restaurant = data[0];

    document.getElementById("restaurant-name").innerText = restaurant.name;
    document.getElementById("restaurant-image").src = restaurant.image;
    document.getElementById("restaurant-image").alt = restaurant.name;
    document.getElementById("restaurant-description").innerText = `Добро пожаловать в ресторан "${restaurant.name}"! Насладитесь ${restaurant.cuisine} кухней в уютной атмосфере.`;
    document.getElementById("restaurant-hours").innerText = restaurant.working_hours;
    document.getElementById("restaurant-address").innerText = restaurant.address;

    const menuList = document.getElementById("restaurant-menu");
    menuList.innerHTML = "";
    restaurant.menu.forEach(item => {
      const li = document.createElement("li");
      li.innerText = item;
      menuList.appendChild(li);
    });
  } catch (error) {
    console.error("Ошибка загрузки ресторана:", error);
    document.getElementById("restaurant-name").innerText = "Ошибка загрузки данных";
  }
}

document.addEventListener("DOMContentLoaded", loadRestaurant);

```

Рис. 3 - Код для отображения данных ресторанов на главной странице

```
1  {
2    "users": [
3      {
4        "id": "1",
5        "name": "Софья",
6        "email": "mymail@mail.ru",
7        "password": "123456",
8        "phone": "880553535"
9      },
10     {
11       "id": "1e29",
12       "name": "Polina",
13       "email": "lobova_p@mail.ru",
14       "telephone": "88005553535",
15       "password": "polina2003"
16     },
17     {
18       "id": "033a",
19       "name": "Кошка",
20       "email": "cat@mail.ru",
21       "telephone": "88005553535",
22       "password": "123456"
23     },
24     {
25       "id": "8989",
26       "name": "поро",
27       "email": "поро@mail.ru",
28       "telephone": "12121212121",
29       "password": "111111"
30     },

```

Рис. 4 - Часть данных JSON файла

[На главную](#)

[Выйти](#)

[Переключить тему](#)

Имя: sobaka

Почта: sobaka@mail.ru

Телефон: +79373622955

История бронирований

• 2025-03-30, 12:12

Ресторан: *La Piazza*

Количество гостей: 12

Статус бронирования: *Ожидание*

Рис. 5 - Отображение данных аккаунта и истории бронирований

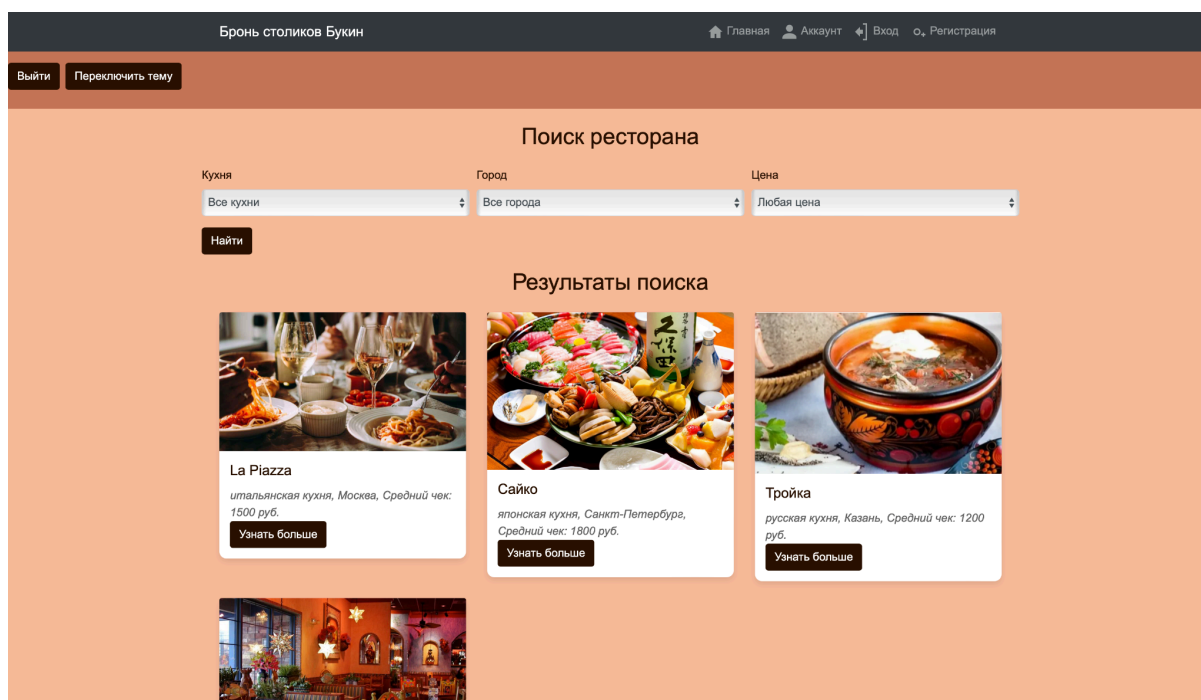


Рис. 6 - Отображение ресторанов на главной странице

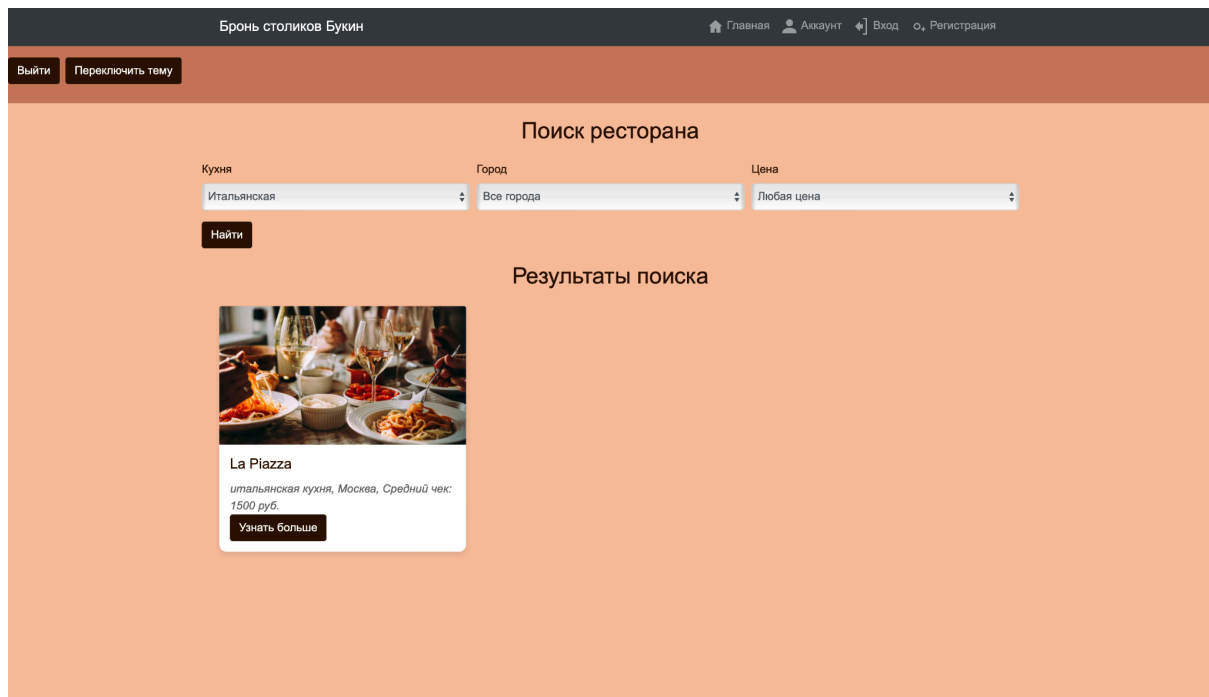


Рис. 6 - Отображение работоспособности фильтров для ресторанов на главной странице

Вывод:

Было реализовано моковое API средствами JSON-сервера. Так, верстка, выполненная в рамках ЛР 1, была привязана к моковому API с помощью `fetch`: теперь пользователь может регистрироваться на сайте, авторизовываться и входить в профиль. В профиле пользователь может посмотреть историю своих бронирований, а если перейдет на главную страницу то может выбрать ресторан с использованием фильтров и забронировать там столик.